NS3 TCP/UDP

Experiment-4

Siddharth Senthilnathan
21BCE1448

Objective

To simulate and analyze single-client communication using TCP and UDP protocols in the ns-3 network simulator, by establishing a connection between a client and a server, transmitting data packets, and observing network behavior such as packet delivery and timing.

Procedure

1. Open the terminal and navigate to the ns-3 directory:
   cd ~/ns-3-dev
2. Move to the scratch folder where simulation files are stored:
   cd scratch
3. Create a new C++ file for the program:
   nano socket-client-server.cc
4. Write the program for single client-server communication using UDP/TCP sockets in ns-3.

5. Save the file and exit the editor.

6. Navigate back to the ns-3 root directory:
   cd ..
7. Compile and run the program using:
   ./ns3 run scratch/socket-client-server.cc
8. Observe the output in the terminal, where the server receives packets sent by the client at regular intervals.

9. Modify the socket type from UDP to TCP by replacing:
   UdpSocketFactory::GetTypeId ()
   with:
   TcpSocketFactory::GetTypeId ()
   and run the program again.

TCP

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

void ReceivePacket (Ptr<Socket> socket)
{
```

```
 Ptr<Packet> packet;
 Address from;

 while ((packet = socket->RecvFrom (from)))
 {
  std::cout << "Server received "
            << packet->GetSize ()
            << " bytes at "
            << Simulator::Now ().GetSeconds ()
            << " s\n";
 }
}

void SendPacket (Ptr<Socket> socket, uint32_t pktSize, uint32_t pktCount, Time pktInterval)
{
 if (pktCount > 0)
 {
  socket->Send (Create<Packet> (pktSize));

  Simulator::Schedule (pktInterval,
                       &SendPacket,
                       socket,
                       pktSize,
                       pktCount - 1,
                       pktInterval);
 }
 else
 {
  socket->Close ();
 }
}

int main (int argc, char *argv[])
{
 Time::SetResolution (Time::NS);

 NodeContainer nodes;
 nodes.Create (2);

 PointToPointHelper p2p;
 p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
 p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

 NetDeviceContainer devices = p2p.Install (nodes);

 InternetStackHelper internet;
 internet.Install (nodes);

 Ipv4AddressHelper ipv4;
 ipv4.SetBase ("10.1.1.0", "255.255.255.0");

 Ipv4InterfaceContainer interfaces = ipv4.Assign (devices);

 uint16_t port = 8080;

 // ----------- SERVER (TCP) -----------
 Ptr<Socket> serverSocket =
   Socket::CreateSocket (nodes.Get (1), TcpSocketFactory::GetTypeId ());

 InetSocketAddress local =
```

```
  InetSocketAddress (Ipv4Address::GetAny (), port);

 serverSocket->Bind (local);
 serverSocket->Listen ();  // IMPORTANT for TCP
 serverSocket->SetRecvCallback (MakeCallback (&ReceivePacket));

 // ----------- CLIENT (TCP) -----------
 Ptr<Socket> clientSocket =
   Socket::CreateSocket (nodes.Get (0), TcpSocketFactory::GetTypeId ());

 InetSocketAddress remote =
   InetSocketAddress (interfaces.GetAddress (1), port);

 clientSocket->Connect (remote);

 Simulator::Schedule (Seconds (2.0),
                      &SendPacket,
                      clientSocket,
                      1024,
                      10,
                      Seconds (1.0));

 Simulator::Stop (Seconds (15.0));

 Simulator::Run ();
 Simulator::Destroy ();

 return 0;
}
```

UDP

```
#include "ns3/core-module.h"
#include "ns3/network-module.h"
#include "ns3/internet-module.h"
#include "ns3/point-to-point-module.h"
#include "ns3/applications-module.h"

using namespace ns3;

// Receive function
void ReceivePacket (Ptr<Socket> socket)
{
    Ptr<Packet> packet;
    Address from;

    while ((packet = socket->RecvFrom (from)))
    {
        std::cout << "Server received "
                  << packet->GetSize ()
                  << " bytes at "
                  << Simulator::Now ().GetSeconds ()
                  << " s\n";
```

```
    }
}

// Send function
void SendPacket (Ptr<Socket> socket, uint32_t pktSize, uint32_t pktCount, Time
pktInterval)
{
    if (pktCount > 0)
    {
        socket->Send (Create<Packet> (pktSize));

        Simulator::Schedule (pktInterval,
                             &SendPacket,
                             socket,
                             pktSize,
                             pktCount - 1,
                             pktInterval);
    }
    else
    {
        socket->Close ();
    }
}

// Main function
int main (int argc, char *argv[])
{
    Time::SetResolution (Time::NS);

    // Create nodes
    NodeContainer nodes;
    nodes.Create (2);

    // Point-to-point link
    PointToPointHelper p2p;
    p2p.SetDeviceAttribute ("DataRate", StringValue ("5Mbps"));
    p2p.SetChannelAttribute ("Delay", StringValue ("2ms"));

    NetDeviceContainer devices = p2p.Install (nodes);

    // Install Internet stack
    InternetStackHelper internet;
    internet.Install (nodes);

    // Assign IP addresses
    Ipv4AddressHelper ipv4;
    ipv4.SetBase ("10.1.1.0", "255.255.255.0");
```

```
    Ipv4InterfaceContainer interfaces = ipv4.Assign (devices);

    uint16_t port = 8080;

    // ---------------- SERVER ----------------
    Ptr<Socket> serverSocket =
        Socket::CreateSocket (nodes.Get (1), UdpSocketFactory::GetTypeId ());

    InetSocketAddress local =
        InetSocketAddress (Ipv4Address::GetAny (), port);

    serverSocket->Bind (local);
    serverSocket->SetRecvCallback (MakeCallback (&ReceivePacket));

    // ---------------- CLIENT ----------------
    Ptr<Socket> clientSocket =
        Socket::CreateSocket (nodes.Get (0), UdpSocketFactory::GetTypeId ());

    InetSocketAddress remote =
        InetSocketAddress (interfaces.GetAddress (1), port);

    clientSocket->Connect (remote);

    Simulator::Schedule (Seconds (2.0),
                         &SendPacket,
                         clientSocket,
                         1024, // packet size
                         10,   // number of packets
                         Seconds (1.0)); // interval

    Simulator::Stop (Seconds (15.0));

    Simulator::Run ();
    Simulator::Destroy ();

    return 0;
}
```

Output:

TCP

```
ns-3-dev — -zsh — 136×34
Last login: Fri Apr 10 04:07:03 on ttys000
siddharthsenthilnathan@Siddharths-MacBook-Pro ~ % cd ns-3-dev/scratch
nano socket-client-server.cc
siddharthsenthilnathan@Siddharths-MacBook-Pro scratch % cd ..
./ns3 run scratch/socket-client-server.cc
[  0%] Building CXX object scratch/CMakeFiles/scratch_socket-client-server.dir/socket-client-server.cc.o
[  0%] Linking CXX executable /Users/siddharthsenthilnathan/ns-3-dev/build/scratch/ns3-dev-socket-client-server-default
Server received 1024 bytes at 2.00369 s
Server received 1024 bytes at 3.00369 s
Server received 1024 bytes at 4.00369 s
Server received 1024 bytes at 5.00369 s
Server received 1024 bytes at 6.00369 s
Server received 1024 bytes at 7.00369 s
Server received 1024 bytes at 8.00369 s
Server received 1024 bytes at 9.00369 s
Server received 1024 bytes at 10.0037 s
Server received 1024 bytes at 11.0037 s
siddharthsenthilnathan@Siddharths-MacBook-Pro ns-3-dev %
```

UDP

```
siddharthsenthilnathan@Siddharths-MacBook-Pro ns-3-dev % ./ns3 run scratch/socket-client-server.cc
Server received 1024 bytes at 2.00369 s
Server received 1024 bytes at 3.00369 s
Server received 1024 bytes at 4.00369 s
Server received 1024 bytes at 5.00369 s
Server received 1024 bytes at 6.00369 s
Server received 1024 bytes at 7.00369 s
Server received 1024 bytes at 8.00369 s
Server received 1024 bytes at 9.00369 s
Server received 1024 bytes at 10.0037 s
Server received 1024 bytes at 11.0037 s
siddharthsenthilnathan@Siddharths-MacBook-Pro ns-3-dev %
```

Conclusion

The experiment was successfully carried out using the ns-3 simulator to implement single-client communication with both TCP and UDP protocols. The client was able to transmit data packets to the server, and the server successfully received them at regular intervals.

UDP communication demonstrated faster transmission due to its connectionless nature, while TCP ensured reliable data delivery through connection-oriented communication. Thus, the experiment highlights the fundamental differences between TCP and UDP in terms of reliability, speed, and connection setup.